

Course Outline: Bases de Datos (Solo Informativo)

Title: Fundamentos de Bases de Datos: Una Introducción Completa **Learnpack:** + Bases de datos (solo informativo) **Prerequisite:** Almacenando información (en cualquier lugar) **Target Audience:** Beginner developers learning database fundamentals and SQL basics **Total Lessons:** 10 **Format:** Mixed (30% hands-on)

Course Description

Este curso proporciona una introducción completa a los fundamentos de las bases de datos, desde conceptos básicos hasta la práctica con SQL. Los estudiantes aprenderán qué son las bases de datos, por qué son esenciales en el desarrollo de software, y explorarán los diferentes tipos de bases de datos disponibles. El curso incluye experiencia práctica escribiendo consultas SQL y comprendiendo las relaciones entre datos.

A través de un enfoque práctico y teórico balanceado, los estudiantes desarrollarán una comprensión sólida de cómo estructurar, consultar y modelar datos usando bases de datos relacionales. Este conocimiento es fundamental para cualquier desarrollador que trabaje con aplicaciones que manejen información persistente.

El curso enfatiza las mejores prácticas de la industria, incluyendo atomicidad de datos, tipado estricto, y patrones de seguridad en consultas, preparando a los estudiantes para trabajar con bases de datos en proyectos del mundo real.

Course Philosophy

- This course emphasizes:
- **Practical understanding** over theoretical complexity
 - **Safety-first approach** to database operations
 - **Industry best practices** from day one
 - **Structured thinking** about data relationships

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Understanding Databases	2
02 - Relational Databases and SQL	3
03 - Advanced Database Concepts	2
04 - Assessment and Mastery	2
05 - Outro	1
Total	10

Section 00: Welcome

00.0 Welcome to Database Fundamentals 📖

Learning Objectives:

- Understand the core problem databases solve in software development
- Recognize how databases fit into the broader web development ecosystem
- Preview the journey from basic concepts to writing SQL queries

Content Outline:

- The data persistence challenge: Why applications need databases
- Preview of database types and when to use each
- Overview of course sections and learning progression
- Connection to previous learning about data storage

Transitions: This welcome sets the foundation for understanding what databases are and why they matter, leading directly into exploring the fundamental concept of databases in the next lesson.

Key Principles:

- Databases solve the persistent data problem that all applications face
- Different types of databases serve different application needs
- Understanding databases is essential for building robust applications

Examples:

- E-commerce site storing product information and user orders
 - Social media platform managing user profiles and posts
 - Banking application handling account balances and transactions
-

Section 01: Understanding Databases

01.0 What Are Databases and Why Do We Need Them?

Learning Objectives:

- Define what a database is and its role in software systems
- Explain the problems databases solve beyond simple file storage
- Identify scenarios where databases are necessary versus optional
- Understand the concept of data persistence and reliability

Content Outline:

- Definition: What constitutes a database system
- The evolution from files to databases: Why the upgrade matters
- Core problems databases solve: Concurrency, reliability, querying, relationships
- Data persistence: Surviving application restarts and crashes
- Real-world scenarios requiring database solutions

Transitions: Now that we understand what databases are and why they're essential, we'll explore the different types available and when to choose each one.

Key Principles:

- Databases provide structured, reliable data storage beyond simple files
- Concurrent access and data integrity are fundamental database features
- The right database choice depends on your application's specific needs
- Data persistence ensures information survives beyond application sessions

Examples:

- Comparing a text file of users vs. a database with user authentication
- Multiple users editing the same document simultaneously

- Application crash scenarios: file corruption vs. database recovery

01.1 Database Types and When to Use Each 📖

Learning Objectives:

- Distinguish between relational and non-relational database types
- Identify the most common database systems and their primary use cases
- Match database types to specific application requirements
- Understand the trade-offs between different database approaches

Content Outline:

- Relational databases (SQL): PostgreSQL, MariaDB/MySQL, SQL Server
- Document databases: TinyDB, SQLite, MongoDB
- Key-value stores: Redis for caching and sessions
- Other specialized types: Graph databases for complex relationships, columnar databases like Cassandra
- Decision matrix: When to choose each type
- Integration with existing tech stacks

Transitions: Having surveyed the database landscape, we'll dive deep into relational databases and SQL, which form the foundation of most web applications.

Key Principles:

- Relational databases excel at structured data with clear relationships
- Document databases offer flexibility for evolving data structures
- Key-value stores provide high-performance caching and simple lookups
- Choose database type based on data structure, relationships, and performance needs

Examples:

- E-commerce: PostgreSQL for orders and inventory, Redis for shopping carts
 - Content management: MongoDB for flexible article schemas
 - Analytics: Cassandra for high-volume time-series data
-

Section 02: Relational Databases and SQL

02.0 Relational Databases: Structure and Principles 📖

Learning Objectives:

- Explain the relational model and how data is organized in tables
- Understand rows, columns, and the concept of primary keys
- Apply the atomicity principle to table design
- Recognize the importance of data types and constraints

Content Outline:

- The relational model: Tables, rows, and columns
- Primary keys: Unique identifiers for each record
- Data types: INTEGER, VARCHAR, BOOLEAN, DATE, etc.
- Atomicity pattern: One cell = one value principle
- Constraints and data integrity: NOT NULL, UNIQUE, CHECK constraints
- Table naming conventions: Plural nouns as best practice

Transitions: With the structural foundation in place, we'll learn the SQL language used to interact with relational databases.

Key Principles:

- Tables represent entities, rows represent instances, columns represent attributes
- Atomicity ensures each cell contains exactly one value
- Strict typing prevents data corruption and maintains consistency
- Primary keys guarantee each record can be uniquely identified

Examples:

- Users table: id, email, created_at columns
- Products table with proper atomic design vs. problematic multi-value cells
- Data type enforcement: INTEGER column rejecting "Hello" input

02.1 SQL Query Fundamentals

Learning Objectives:

- Write basic SELECT statements to retrieve data
- Use WHERE clauses to filter results effectively
- Apply ORDER BY and LIMIT for result organization
- Understand explicit projection: requesting only needed columns

Content Outline:

- SELECT statement anatomy: SELECT, FROM, WHERE structure
- WHERE clause operators: =, !=, >, <, LIKE, IN, BETWEEN
- Sorting results with ORDER BY (ASC/DESC)
- Limiting results with LIMIT and OFFSET
- Explicit projection pattern: Selecting specific columns vs. SELECT *
- String pattern matching with LIKE and wildcards

Transitions: Now we'll put these SQL concepts into practice by writing and executing real queries.

Key Principles:

- Explicit projection improves performance and clarity
- WHERE clauses should be specific to avoid unintended results
- Always test queries with SELECT before using UPDATE or DELETE
- Sort and limit results to make data manageable

Examples:

- Finding users by email domain: WHERE email LIKE '%@company.com'
- Getting the 10 most recent orders: ORDER BY created_at DESC LIMIT 10
- Explicit vs. implicit column selection performance comparison

02.2 Writing Your First SQL Queries

Challenge Prompt: Write SQL queries to retrieve, filter, and organize data from a products database, demonstrating proper SELECT syntax, WHERE filtering, and result ordering.

Solution Code:

```
-- Basic product retrieval
SELECT name, price, category FROM products WHERE price > 50;

-- Products in specific categories, ordered by price
SELECT * FROM products
WHERE category IN ('Electronics', 'Books')
ORDER BY price ASC
```

```
LIMIT 5;

-- Pattern matching for product names
SELECT name, price FROM products
WHERE name LIKE '%phone%'
AND price BETWEEN 100 AND 500;

-- Count products by category
SELECT category, COUNT(*) as product_count
FROM products
GROUP BY category
ORDER BY product_count DESC;
```

Challenge Code:

```
-- Complete these SQL queries for the products table
-- Table structure: id, name, price, category, stock_quantity, created_at

-- 1. Select all products with price greater than 100
SELECT

-- 2. Find products containing 'laptop' in the name, ordered by price
SELECT

-- 3. Get the 3 most expensive products in 'Electronics' category
SELECT

-- 4. Count how many products exist in each category
SELECT
```

Section 03: Advanced Database Concepts

03.0 Relationships Between Tables and Data Modeling 📖

Learning Objectives:

- Design normalized database schemas with proper relationships
- Understand one-to-many, many-to-many, and one-to-one relationships
- Create and use foreign keys to maintain referential integrity
- Apply normalization principles to eliminate data redundancy

Content Outline:

- Database normalization: Why and how to structure related data
- Relationship types: One-to-many (users → orders), many-to-many (students ↔ courses)
- Foreign keys: Connecting tables through references
- Junction tables: Solving many-to-many relationships
- Referential integrity: Ensuring data consistency across relationships
- Denormalization considerations: When to break normalization rules

Transitions: With relationship concepts established, we'll practice writing complex queries that work across multiple related tables.

Key Principles:

- Normalize data to eliminate redundancy and update anomalies
- Foreign keys maintain data integrity across table relationships

- Junction tables handle complex many-to-many relationships
- Each table should represent a single entity or concept

Examples:

- E-commerce schema: users, orders, products, order_items tables
- Blog system: authors, posts, categories, tags with proper relationships
- School database: students, courses, enrollments junction table

03.1 Complex SQL Operations and Joins

Challenge Prompt: Write JOIN queries to retrieve related data from multiple tables, demonstrating INNER JOIN, LEFT JOIN, and aggregate functions across relationships.

Solution Code:

```
-- Users with their order count
SELECT u.email, u.name, COUNT(o.id) as order_count
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
GROUP BY u.id, u.email, u.name
ORDER BY order_count DESC;

-- Products sold with quantities
SELECT p.name, SUM(oi.quantity) as total_sold
FROM products p
INNER JOIN order_items oi ON p.id = oi.product_id
GROUP BY p.id, p.name
HAVING SUM(oi.quantity) > 10;

-- Recent orders with user and product details
SELECT o.created_at, u.email, p.name, oi.quantity
FROM orders o
JOIN users u ON o.user_id = u.id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
WHERE o.created_at > '2024-01-01'
ORDER BY o.created_at DESC;
```

Challenge Code:

```
-- Write JOIN queries using these tables:
-- users(id, email, name)
-- orders(id, user_id, total, created_at)
-- order_items(id, order_id, product_id, quantity)
-- products(id, name, price, category)

-- 1. Get all users and their total number of orders (including users with 0 orders)
SELECT

-- 2. Find products that have never been ordered
SELECT

-- 3. Get the top 5 users by total money spent
SELECT
```

Section 04: Assessment and Mastery

04.0 Database Best Practices and Common Pitfalls 📖

Learning Objectives:

- Apply the "safe mode" pattern for destructive database operations
- Implement proper naming conventions and schema design principles
- Recognize and avoid common database anti-patterns
- Understand performance considerations for query optimization

Content Outline:

- Safe mode pattern: SELECT before UPDATE/DELETE operations
- Naming conventions: Plural table names, consistent column naming
- Schema design best practices: Appropriate data types, proper indexing considerations
- Common anti-patterns to avoid: Storing multiple values in single cells, missing foreign key constraints
- Performance awareness: Query optimization basics, avoiding SELECT * in production
- Data backup and recovery considerations

Transitions: Let's test your comprehensive understanding of database fundamentals through a practical assessment.

Key Principles:

- Safety first: Always verify with SELECT before modifying data
- Consistency in naming and structure improves maintainability
- Atomic data design prevents complex parsing and ensures data integrity
- Performance considerations should guide schema and query design

Examples:

- Safe UPDATE: First `SELECT * FROM users WHERE id = 5`, then `UPDATE users SET email = 'new@email.com' WHERE id = 5`
- Anti-pattern: Storing "Red, Blue, Green" in single cell vs. separate color table
- Performance comparison: `SELECT id, name FROM users` vs. `SELECT * FROM users`

04.1 Database Fundamentals Assessment 💡

Learning Objectives:

- Demonstrate understanding of database types and their appropriate use cases
- Apply SQL query knowledge to solve real-world data retrieval problems
- Identify proper database design patterns and avoid anti-patterns
- Show mastery of relationship modeling and join operations

Assessment Questions (Multiple Choice):

1. Which database type is BEST suited for storing user session data that needs to be accessed very quickly?

- A) PostgreSQL (relational)
- B) MongoDB (document)
- C) Redis (key-value)
- D) Cassandra (columnar)

Correct: C - Redis excels at fast key-value lookups for session data

2. What does the atomicity principle mean in database design?

- A) All operations must complete or none at all
- B) Each cell should contain exactly one value
- C) Tables must have primary keys

- D) Queries should be as fast as possible

Correct: B - One cell = one value is the atomicity pattern for table design

3. Before running `DELETE FROM users WHERE age > 65`, what should you do first?

- A) Back up the entire database
- B) Run `SELECT * FROM users WHERE age > 65`
- C) Check if other tables reference these users
- D) Both B and C

Correct: D - Safe mode requires verification and checking dependencies

4. Which SQL query demonstrates explicit projection best practices?

- A) `SELECT * FROM products`
- B) `SELECT name, price FROM products WHERE category = 'Books'`
- C) `SELECT products FROM inventory`
- D) `SELECT COUNT(*) FROM products`

Correct: B - Selects only needed columns with proper filtering

5. In an e-commerce database, how should you model the many-to-many relationship between products and orders?

- A) Store product IDs as comma-separated values in the orders table
- B) Store order IDs as an array in the products table
- C) Create an order_items junction table with order_id and product_id
- D) Duplicate product information in each order

Correct: C - Junction tables properly handle many-to-many relationships

6. What's the PRIMARY advantage of relational databases over simple file storage?

- A) They take up less disk space
- B) They're easier to learn and use
- C) They handle concurrent access and maintain data integrity
- D) They work faster for all types of data

Correct: C - Concurrency and integrity are key database advantages

7. Which naming convention follows database best practices?

- A) Table: `User`, Column: `UserEmail`
- B) Table: `users`, Column: `email`
- C) Table: `tblUsers`, Column: `strEmail`
- D) Table: `USERS`, Column: `EMAIL_ADDRESS`

Correct: B - Plural table names and simple column names are standard

Section 05: Outro

05.0 Your Database Journey Forward

Content Outline:

- **Course recap:** You've learned what databases are, explored different types (relational, document, key-value), mastered SQL fundamentals, and practiced designing relationships between data. You can now make informed decisions about database selection and write effective queries.

- **Connection to bigger picture:** Database knowledge forms the foundation for backend development, data analysis, and system architecture. These concepts apply whether you're building web applications, mobile backends, or data processing systems.
 - **Next steps and continued learning:**
 - Practice with real database projects and larger datasets
 - Explore advanced SQL features like stored procedures and triggers
 - Learn database administration and performance optimization
 - Study NoSQL databases in depth for specific use cases
 - Consider database design patterns for microservices architectures
 - **Resources for further exploration:**
 - PostgreSQL Tutorial (official documentation)
 - SQLBolt for interactive SQL practice
 - Database design books like "Database Design for Mere Mortals"
 - Online SQL practice platforms like HackerRank and LeetCode
 - MongoDB University for NoSQL learning
 - **Encouragement and celebration:** Congratulations! You've built a solid foundation in database fundamentals. Understanding how to structure, query, and model data is a crucial skill for any developer. These concepts will serve you throughout your programming career, whether you're building simple web apps or complex distributed systems. Keep practicing with real projects, and don't hesitate to experiment with different database types as you encounter new challenges.
-